

Speaker Notes

Contents

- [1 1. I/O from Python to the operating system](#)
- [2 2. Where we stopped](#)
- [3 3. Roadmap](#)
- [4 4. Takeaways](#)
- [5 5. What I cover today](#)
- [6 6. What I leave out today](#)
- [7 7. What does one byte cost?](#)
- [8 8. What does one byte cost?](#)
- [9 9. Two programs, two access patterns](#)
- [10 10. Predict before we run](#)
- [11 11. The measurement runs here, not on a slide](#)
- [12 12. Your laptop will not agree with this projector](#)
- [13 13. Where does the difference come from?](#)
- [14 14. Where does the difference come from?](#)
- [15 15. What one crossing costs](#)
- [16 16. `strace` : the tool that watches the boundary](#)
- [17 17. What one trace line says](#)
- [18 18. Count the crossings](#)
- [19 19. The count ratio over-predicts the time](#)
- [20 20. What a buffer actually does](#)
- [21 21. Why a seek gives the saving back](#)
- [22 22. Does the seek itself cost a crossing?](#)
- [23 23. Trust the counts. Do not trust the times.](#)
- [24 24. Windows counts too — differently](#)
- [25 25. Counting on Windows: ask the kernel](#)
- [26 26. What the counters will not tell you](#)
- [27 27. It is not one buffer — it is a stack](#)
- [28 28. It is not one buffer — it is a stack](#)
- [29 29. Four levels, and who empties each one](#)
- [30 30. To flush is to cross one boundary](#)
- [31 31. Wholesale down, retail up](#)
- [32 32. One program, one buffer — C and Python take different paths](#)
- [33 33. The bottom level acknowledges early too](#)

34 34. Three lines that print nothing

35 35. Why the file is empty

36 36. What this demonstration does not prove

37 37. `ltrace` : the same idea, one layer up

38 38. Which levels can you actually watch?

39 39. The same write, at three levels

40 40. The same write, at three levels

41 41. Meet the program: `iodemo.c`

42 42. Three calls in the source. One in the trace.

43 43. `fwrite` takes a size and a count

44 44. Two operating systems, side by side

45 45. Process Monitor: files and registry, not system calls

46 46. NtTrace: a tracer for the Windows Native API

47 47. Why NtTrace and not Process Monitor

48 48. Three Windows words you will need

49 49. A real Windows trace

50 50. A handle is not a file descriptor

51 51. Two error namespaces in one return value

52 52. The stable boundary sits at a different level

53 53. Where the boundary sits is contested history

54 54. Linux is the outlier, not Windows

55 55. One program, two kernels

56 56. What we have not looked at

57 57. `write()` returned success. The bytes are not on a disk.

58 58. Buffering moves your errors

59 59. Never retry a failed close

60 60. Networking and IPC are this same mechanism

61 61. Honest limits

62 62. Honest limits, continued

63 63. What to do on Monday

64 64. Summary — what follows

65 65. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Slides: 65 · **Duration:** 62 min · **Venue:** Faculty of Informatics, Masaryk University (FI MUNI)

11. I/O from Python to the operating system

On screen:

PB158 · Lecture 2

The call you write is not the call the operating system runs [inference]. `fflush()` flushes only the C library's buffer [S3]. The kernel buffer needs a second call [S3]. Four boundaries sit below your code: app to library, library to kernel, kernel to device, and device to medium. A different tool watches each of the first three [V10] [S1] [V30]. No tool here watches the fourth [V5].

*Ondřej (Ondra) Krajíček me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek
Built at render time*

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

Welcome back. Lecture one was about how this course works. This one is the first technical lecture and it runs for about an hour.

Let me set the frame in one sentence. You write Python and you write C. Your theory course told you what a system call is. Most of you have never watched one happen [speculative]. That is the gap I close today, and I close it with a measurement rather than a definition.

Three programs run live on this machine. If a demonstration fails, every claim I rely on is also written on a slide. Nothing in the argument depends on the projector.

One housekeeping note. Everything written in this course is in English, including the slides and the code comments. I speak Czech. The written English is deliberately plain so you can read it while listening to me.

2 2. Where we stopped

On screen:

Last time · Lecture 1

1

[P1] The course grades looking

Not reciting. Ondra built the assessment around what a student can show.

2

[P2] The course works downward

From the code you write to the operating system under it.

Ondra asserted both in Lecture 1. This hour is where he makes the first one good.

Two claims came out of the orientation lecture, and both return in this hour.

The first is about grading: this course asks what a system does, not what a textbook says about it. Today that becomes one measurement and one trace.

The second is the direction of travel, and this lecture is the first descent.

3 3. Roadmap

On screen:

Roadmap

1

What does one byte cost?

A stopwatch.

2

Where does the difference come from?

A tracer.

3

It is not one buffer — it is a stack

A diagram.

4

The same write, at three levels

A program.

5

Two operating systems, side by side

Two traces.

6

What we have not looked at

One open question.

Here is the shape of the hour, so you know where the payoff sits.

Sections one and two are twenty minutes on a single question. Why do two programs that write the same bytes take very different times? The answer is a number you can count, and by the end of section two you will count it yourself.

Section three is the structural part. I draw one diagram there, and I point back at it in the durability lecture, the memory mapping lecture and the crash analysis lecture.

In sections four and five I spend the tracer, first on one program at three levels, then on two operating systems. In section six I deliberately leave one question open, and that question is the bridge into the rest of the course.

Assignment one measures what section one measures, so you will have the model to explain your own numbers.

4 4. Takeaways

On screen:

Where we are going

1

[T1] Adjacency decides the call count

The saving is in calls, not work, and only for adjacent bytes [inference]. A seek empties the buffer [speculative].

2

*[T2] A library call is not a system call**Several levels sit beneath yours [S3].*

3

*[T3] The boundary is observable**A tracer shows what your line became [S1].*

4

*[T4] The platforms differ by design**Each nominates a different level [V8].*

These four claims are what I want you to hold in a week. I show them now so you know what each section is for.

Number one is the one you use on Monday, and it is what assignment one measures. Notice that it is not the rule "buffering is fast". It carries a condition, and the condition is the interesting part. Lampson put sequential access in the same sentence as his batching advice in 1983 [S12]. The seek half is mine, argued in part two [speculative].

Number two is the structural claim, and it is why I am giving this lecture.

Number three is the promise about the instrument. By the end of section two you will read a trace line, and you will know which parts of it to believe.

Number four is the platform claim. Some of you run Windows. I am not going to tell you that one system is right. I am going to show you that they nominate different levels as stable.

I bring all four back at the end, and each section earns one of them.

5 5. What I cover today

On screen:

Scope

1

Why two programs differ

2

When a buffer helps

3

The four levels

4

Reading a trace

What is on this slide is what assignment one measures. Everything I leave out comes on the next slide, with a lecture number attached.

6 6. What I leave out today

On screen:

Scope

5

When a write is safe

6

Memory mapping

7

Operating the tools

8

Asynchronous I/O

I want to fence the hour, because three of these exclusions are the ones you will most want to ask about.

The first is durability. I will show you today that "a successful return from write() does not make any guarantee that data has been committed to disk" [S2]. I will not tell you today what does, because today's instrument cannot answer it. Today's trace stops at the system call boundary, and no trace separates a device that flushed from one that only reported it [V5]. I answer it in the durability lecture, with a different method.

The second is running the tools. You have no toolchain installed yet, and lecture one told you to install it before the first lab. So today you read traces and I run them.

The third is asynchronous I/O. It changes the picture, and I say one sentence about it near the end.

Everything here has a lecture number attached, and I name each one as we pass it.

7 7. What does one byte cost?

On screen:

Part I

Two programs. The same mebibyte. Predict the ratio before we run them.

Nine minutes, and this part contains no theory at all. It contains two programs and a stopwatch.

The order is deliberate. The usual way to teach this is to explain the buffer first and then show a benchmark that confirms it. That teaches you to trust the explanation. I want you to see the number first, decide you cannot explain it, and then want the explanation.

Write your prediction down. Do not keep it in your head.

8 8. What does one byte cost?

On screen:

Part I

Two programs write the same mebibyte to a file. That is 1,048,576 bytes. One uses a buffer. One does not.

[code]

One argument differs. Python documents it: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

[S6] Built-in Functions, open() — Python documentation. <https://docs.python.org/3/library/functions.html#open>

Read both programs now. The next twenty minutes depend on you believing they are the same program apart from one argument [S6].

Same loop. Same iteration count. Same interpreter. Same bytes, in the same order, to the same kind of file.

Notice also that you have to ask for the unbuffered version. It is not the default. You cannot write program A by accident, and that will matter when we decide what the practical lesson is.

One word about the word buffer, because I use it in exactly one sense today. A buffer is memory that collects bytes for consecutive positions in the file. The next level then takes them in one call instead of many. It saves calls. It does not save work.

9 9. Two programs, two access patterns

On screen:

Part I

Sequential access

Each write continues where the last one ended [inference].

Random access

Each write follows a move to a new offset [inference].

Why it matters

Lampson advised batch processing in 1983. He set sequential access aside from that advice, with the words "even aside from" [S12].

[S12] Hints for Computer System Design — Lampson (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>

Two terms, and I hold them to these meanings for the rest of the course.

Sequential access means each write continues where the last one ended. Random access means each write follows a move to another offset. Do not say scattered and do not say non-linear. Say random access, because that is what the assignment says.

Both programs on the last slide use sequential access. Assignment one asks you to run the comparison a second time with random access. It also asks you to explain the difference. Listen to section two with that in mind.

The vocabulary is not new. Lampson wrote both terms down in 1983, in a paper of design hints [S12]. Use batch processing if you can, he says, because doing things incrementally almost always costs more. That reason ends with a clause — even aside from the fact that disks and tapes work much better when accessed sequentially [S12]. Those three words set the clause apart, and he does not rest the advice on it.

That sentence is forty-three years old, and it names the two access patterns you measure next week.

10 10. Predict before we run

On screen:

Part I

Write down a ratio. How much slower is the unbuffered program?

1×

nothing worth measuring

10×

an optimisation

100×

a different kind of thing

Writing a number down before the run changes what you do with the result. Taber calls the method predict-observe-explain, and reports that "making a prediction will give students some sense of ownership in the following activity" [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Pick one of the three numbers. Write it on paper. Take thirty seconds [S14].

I will ask for a show of hands, and I want you to decide before you see the answer. This is not a game. The method has a name, and Taber describes it for exactly this case, where a room holds a strong intuition the machine does not match [S14].

Hands up for one times. Hands up for ten times. Hands up for one hundred.

Now I run both programs on this machine and we see which hand was closest.

Whatever number appears is a fact about this machine, this Python version and this filesystem. Your number will differ, and that turns out to be the lesson rather than a nuisance.

11 11. The measurement runs here, not on a slide

On screen:

Part I

I print no ratio here, on purpose. A benchmark you did not watch me produce is the kind of claim this course teaches you to distrust.

Held fixed

Loop, interpreter, iteration count, bytes, filesystem [inference].

Varied

One argument, buffering=0 [S6].

Write down

Both times, the Python version, the buffer size [inference].

What it includes

The operating system's own memory for file data, in both runs. Name the cache layer your number includes, because you cannot remove all of them [V19].

I am running both programs now, untraced, with a plain wall-clock timer.

While they run, let me be honest about what this measures. One mebibyte does not reach a disk in any way you could detect. It lands in the operating system's own memory and stays there. So this measures the cost of asking the operating system to do something, a million times against a few hundred. Name the cache layer your number includes, because you cannot remove all of them [V19].

Copy both times into your notebook. You need them for assignment one, and the review lecture on the sixth of October uses your numbers rather than mine.

Now look at the ratio. Then look at the hands that went up. The reaction I am after is that this stopped being an optimisation. I built part one to produce it.

12 12. Your laptop will not agree with this projector

On screen:

Part I

The buffer size is not a constant. It grew sixteen-fold in Python 3.14.

Python 3.13

The default "was set to 8192 since 16 years ago" [S7].

Python 3.14

"the buffer will typically be 128 kilobytes long" [S6].

Why

Write performance improved "3 to 5 times" over the 8192-byte default [S7]. The reporter measured it. No independent reproduction [inference].

So

Report your version and buffer size with every number [inference].

[S6] Python documentation, `open()`. [S7] CPython issue 117151. <https://github.com/python/cpython/issues/117151>

I refuse to print a benchmark figure, and this slide is why.

Python's default buffer was eight kilobytes for sixteen years, and is now typically one hundred and twenty-eight [S6] [S7]. The change landed in Python 3.14. The project's own issue explains the old value [S7]. The maintainers chose eight kilobytes in 2008. The machine they chose it for ran a thirty-two bit operating system with under two gigabytes of memory [S7].

One caution on that multiplier. The issue author measured it himself, on Python 3.11, writing a file he had downloaded over HTTP [S7]. A core developer confirmed the direction in the same thread, and posted no numbers [S7]. So nobody reproduced the figure [inference]. I quote it as the maintainers' stated reason for the change, not as a figure you can rely on.

Think what that does to your homework. Say some of you run 3.13 and others run 3.14. Your crossing counts then differ by a factor of sixteen. That reason has nothing to do with what the assignment measures. So report the version and the buffer size beside every number. One line of Python prints it.

13 13. Where does the difference come from?

On screen:

Part II

One answer, stated immediately, and then the instrument that proves it.

Twelve minutes. This part explains the number you just watched, and it introduces the instrument the rest of the course depends on.

I will tell you the answer immediately rather than build up to it. The answer is short and the interesting work is in the consequences.

First one elimination. It is not the disk. One mebibyte never reached a disk in either run. It is not the interpreter either, because both runs share it. So whatever the difference is, it happens between your loop and the operating system.

14 14. Where does the difference come from?

On screen:

Part II

The difference is the number of times each program crosses into the operating system.

*Your theory course defined a system call. Today I count them: each one makes a **cross-ing** out of your program, "a privilege transition, not a jump" [V15].*

A

Unbuffered

One call per byte, so 1,048,576 calls [inference].

B

Buffered

One mebibyte divided by the buffer size [inference].

=

So measure first

At 8 KiB, 128 calls. At 128 KiB, 8 calls [inference].

Here is the answer. The two programs differ in the number of crossings they make into the operating system [inference].

Program A asks once per byte. A mebibyte is one million and forty-eight thousand five hundred and seventy-six bytes, so that many requests.

Program B asks once per buffer fill. Divide one mebibyte by the buffer this machine allocated. Print the size before you divide, because I am not going to guess it for you.

Be precise about the vocabulary now, because I use it for the rest of the course. You already know what a system call is, so I am not going to define it. I am going to name the thing I count. The passage out of your program and back is one crossing, and every crossing is one system call.

Do not call it a kernel call. Do not call it an API call. An API call may or may not become one, and that distinction is what I am here to draw.

15 15. What one crossing costs

On screen:

Part II

4,972

instructions, one stat call

13,585

cycles for the same call

2,559

L3 lines evicted — your program was running [inference]

Soares and Stumm measured this on 2010 hardware [S16]. The counts are not current [V44]. What carries over is the regime distinction, not the numbers [inference]. That distinction is a call rate. The lost cache state dominates at one call per 2,000 to 100,000 instructions. The transition cost takes over above that rate [S16].

[S16] FlexSC — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf

Why is a crossing expensive? Your theory course probably said the processor changes privilege level. That is true, and it is not the main cost [S16].

Soares and Stumm measured what one system call does to the processor. Retiring an instruction means finishing it, and one `stat` call retires 0.37 instructions per cycle [V2]. That is roughly one finished instruction every three cycles [inference]. Straight-line code on the same processor finishes more than one per cycle, so this is slow work rather than ordinary work [speculative].

Now the third number. That one call evicted two and a half thousand lines from the level three cache, the processor's own last cache before main memory [V2]. The paper counts the lines. It does not say whose data they held, and your program was the code running [inference]. Every one of those lines your code touches again is a miss it pays for after the call returns [inference].

Two caveats. The measurement is from 2010 hardware, so do not quote the counts as current. And the paper separates two regimes by call rate [S16]. In the medium band, one call per two thousand to one hundred thousand instructions, the lost cache state dominates. Above that rate the transition itself takes over [S16].

Program A makes one system call per loop iteration, so it probably sits above that band [speculative]. That placement is mine, and I have no count of the interpreter's instructions per call [speculative].

Either way the cost is largely fixed per call. Doubling the bytes in one call is nearly free. Doubling the calls is not.

16 16. `strace` : the tool that watches the boundary

On screen:

Part II

A tracer runs your program and writes down its system calls [S1] [T3].

What it does

It "intercepts and records the system calls made by a process" [S1]. It never samples [inference].

Where it comes from

Paul Kranenburg wrote the original for SunOS. Branko Lankester ported it [S1]. The licence is the Lesser GPL [S30].

How to use it

[code]

It is a Linux tool [S30]. It costs speed [S1]. I use NtTrace in Part V [S10].

*[S1] strace(1) man page. <https://man7.org/linux/man-pages/man1/strace.1.html> [S30]
strace. <https://strace.io/> [S10] NtTrace. <https://rogerorr.github.io/NtTrace/>*

You cannot read a trace until you know what produced it [speculative].

What it does. Its manual page says it intercepts and records the system calls a process makes, and the signals it receives [S1]. So it sits at the boundary and records the crossings [inference]. It records rather than samples, which is why you may believe its counts [inference].

The project page calls it "a diagnostic, debugging and instructional userspace utility for Linux" [S30]. It runs on Linux only [inference]. The manual's history says that in 2012 "unmaintained and apparently broken support for non-Linux operating systems was removed" [S1].

Where it comes from, because somebody built this tool. Paul Kranenburg wrote the original for SunOS, and Branko Lankester ported it to Linux [S1]. The licence is the Lesser GPL, and the repositories are public [S30]. It is already in the course dev container, a prepared Linux environment VS Code starts for you. In the lab you run it yourselves.

What it costs. The manual page lists this under BUGS: a traced process runs more slowly than a non-traced one [S1]. Figures come later. Never benchmark under the tracer.

How you use it. `iodemo` in those commands is the small C program we compile in part four, and the same forms work on today's Python pair. Plain `strace` gives one line per call, and we read one on the next slide. `-c` gives a table instead, and `-e trace=write` keeps only the writes [S1].

17 17. What one trace line says

On screen:

Part II

A trace records the system calls a program made, in order, with arguments and return values.

[code]

1

The name

Which system call ran [S1].

2

The arguments

In parentheses, decoded [S1].

3

The return value

After the equals sign [S1].

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did [inference].

[S1] strace(1) — Linux man-pages, which documents this example line. <https://man7.org/linux/man-pages/man1/strace.1.html>

This is one line of its output, and it is the example the manual page itself uses [S1].

Read it left to right. The name of the system call. Then the arguments in parentheses, decoded into names rather than raw numbers. Then an equals sign and the return value.

That is the whole format. Every line looks like this. Once you can read one line you can read a thousand, and the rest of this course assumes you can.

One word about vocabulary. A trace is not a log. A log contains what the author decided to print. A trace contains every call the program made, including the ones the author never printed. So a tracer answers questions the author did not anticipate.

Read that line again, slowly.

18 18. Count the crossings

On screen:

Part II

The same two programs, run again under a counting tracer.

Command

strace -c -e trace=write python3 a.py [S1]

Program A

One write per byte [inference].

Program B

One write per buffer fill [inference].

First

Print `io.DEFAULT_BUFFER_SIZE`, then divide [S6].

Two runs

A traced run cannot be timed [V17].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>

I am running the counting version now. The filter matters. Before your code starts, Python itself opens and reads a large number of files. Without the filter you get every call that work makes, and the one interesting line disappears into the noise [S1].

Look at the two summaries side by side. First I print the buffer size this machine uses. Then I divide one mebibyte by it. Then I compare my result with the count the tool reported. If those two disagree, the arithmetic used the wrong divisor. Finding that out is the exercise working rather than failing.

This is what I mean when I say the boundary is observable. You run one command and you get the exact number of requests your program made. Not an estimate, and not a model. Program A was slow because it made a million of them.

19 19. The count ratio over-predicts the time

On screen:

Part II

Illustration, not a measurement from this room: a count ratio of 100,000 beside a time ratio of 40 is not a mistake [inference].

$time \approx crossings \times fixed\ cost + bytes \times marginal\ cost$ [V1]

1

Two terms

Not a function of bytes alone [V1].

2

One term is shared

Both programs write the same mebibyte through the same loop [inference].

3

So counts over-predict

Under this model the count ratio exceeds the time ratio [inference].

I added this slide because of a sentence I was tempted to say and will not. The tempting sentence is that the ratio of the counts is the ratio of the times. It is false, and the run you just watched shows it false [V1].

Now apply it. Both programs write one mebibyte through the same million-iteration loop. So the second term, and all the interpreter work, is identical. It is a floor neither program can get under.

Under that model the count ratio over-predicts the time ratio, and the shared floor is why. If your counts differ by a hundred thousand and your times by forty, you measured the floor rather than making a mistake. Those two figures are an illustration, not a measurement from this room.

One caveat on the model itself. My note grades it confident rather than verified [V45]. And the per-crossing cost is not one number. One reported measurement spans 76 to 668 cycles for the same calls, across fifteen hosts [V46]. Treat the model as a shape, not as a formula to solve. You will meet this gap again in assignment one.

20 20. What a buffer actually does

On screen:

Part II

A buffer collects adjacent bytes and hands them down in one call.

[code]

Cost has two terms: the number of crossings, and the number of bytes [V1]. A buffer cuts the first and leaves the second alone, so the saving sits entirely in the call count [inference].

Look at the picture. Your loop still runs a million times. Every one of those writes still happens, and every byte is still copied [inference].

What changed is the right-hand arrow. Instead of a million crossings there are a few hundred, because the buffer holds bytes until it is full and then hands over a block.

I want to be precise about the word adjacent, because the next slide turns on it. A buffer holds bytes destined for consecutive positions in the file. It can do that because a block write to the operating system carries one offset and one length. Bytes going to one place travel together. Bytes going to two places cannot.

Say it back to yourselves in one sentence. A buffer saves calls, not work.

That is takeaway one, half stated. The other half is next, and it is the half the assignment measures.

21 21. Why a seek gives the saving back

On screen:

Part II

A buffer holds bytes for consecutive positions. Move elsewhere and those bytes cannot stay in it.

1

Constraint

Buffered bytes belong at the old offset [inference].

2

Consequence

Every seek forces the buffer out [inference].

3

Result

The buffered run converges on the unbuffered [inference].

The rule is buffering is fast when writes are adjacent, and buys little when they are not [inference] [T1]. Lampson gave the batching half in 1983 [S12].

If you keep one slide from today, keep this one. Assignment one is graded on the rule it states.

A buffer collects bytes for consecutive positions. Now suppose you write one byte, move to a different offset, and write another. The first byte belongs at the old offset. It cannot sit in the buffer waiting, because the buffer has one destination. So the library must hand it over before it can move [inference].

Every seek forces the buffer out [inference]. Seek between every write and the buffer never holds more than one byte. The buffered program then makes the same number of crossings as the unbuffered one, and the trace shows it.

So the rule has a condition. Buffering is fast when writes are adjacent, and buys little when they are not. In 1983 Lampson wrote the batching half [S12]. The part about a seek emptying a userspace buffer is mine [inference]. It follows from the mechanism above rather than from him [inference].

One honest limit. Adjacency is necessary and not sufficient. A program already writing a mebibyte in one call has nothing left to collect.

22 22. Does the seek itself cost a crossing?

On screen:

Part II

Yes. And a pair of system calls exists to remove it.

Naive

lseek then write — two crossings for one operation [inference].

Merged

pread and pwrite "combine[s] file access and seek operations into a single command" [V3].

Measured

The merged pair cut the call count [V3]. For the same data volume the study also reports "a relatively reduced load in terms of overall duration" [V27].

Somebody always asks this, so I answer it before you do [speculative].

Yes. Moving to a new offset is itself a request to the operating system. So a random-access run does not make one crossing per byte. It makes two.

That is worth knowing for your assignment, because the random pattern is worse than the sequential unbuffered pattern rather than merely equal to it.

And here is the evidence that the crossing count is the real variable. Linux offers a pair of calls that merge the move and the access into one call. A study replaced the two-call pattern with the merged pair. The call count fell [V3]. For the same data volume the study reports a reduced overall duration [V27].

I am not claiming that the crossing count was why those calls were added. I do not know that, and the source does not say it. My claim is narrower. Merging two crossings into one gave a relatively reduced overall duration for that workload [V27].

You will not need these calls for assignment one. I mention them as evidence.

23 23. Trust the counts. Do not trust the times.

On screen:

Part II

The tracer stops your program twice per call. That destroys the timing and leaves the counting intact.

EVIDENCE	WHAT IT SAYS	TAKE IT AS
<i>Every call stops</i>	<i>so "the counts are trustworthy and the absolute times are not" [V13]</i>	<i>counts only</i>
<i>Gregg's dd loop</i>	<i>one byte per call, "runs 442× slower under strace" [V17]</i>	<i>his machine</i>
<i>Re-runs</i>	<i>"roughly 37× to 136×", other hardware [V31]</i>	<i>not a bound</i>

The range does not contain Gregg's figure. Each number measures one machine and one workload [inference].

You have no reason yet to believe this tracer. Let me give you one. Let me also tell you which of its numbers you must not trust.

Because it stops every call that reaches the kernel, it counts every one of them. Nothing is sampled. That is why you may believe the counts.

One program is allowed to freeze another at each call, read its state, and let it continue [inference]. That is the whole mechanism here.

There is one gap worth knowing. Linux serves a few calls without a crossing at all, from a page it shares with your process. The tracer is silent on those, because there is nothing to intercept [V26].

Because it stops every call, it also makes your program much slower. Brendan Gregg's worst case is a dd loop copying one byte per call, which does almost nothing but system calls. It ran four hundred and forty-two times slower under the tracer [V17]. Later runs of that same loop report thirty-seven to one hundred and thirty-six times, on other hardware and with different processor security mitigations [V31]. Notice that the later range does not contain Gregg's figure.

None of the numbers is wrong, because the overhead follows the call rate of one program on one machine [inference]. A single overhead number is therefore not a fact about the tool [inference].

So the rule for this course is: count under the tracer, time without it. That is why part one and part two used two different runs, and I said so at the time.

If you benchmark under a tracer in the lab, you will report nonsense.

24 24. Windows counts too — differently

On screen:

Part II

NtTrace, the Windows tracer in this course, prints no counting summary [speculative]. Both systems count for you anyway.

PLATFORM	WHERE THE COUNT LIVES	WHAT THE FIELD IS DEFINED AS
Windows	<code>GetProcessIoCounters</code> fills <code>IO_COUNTERS</code> [S24]	<code>WriteOperationCount</code> — "The number of write operations performed" [S23]
Linux	the file <code>/proc/pid/io</code> [S25]	<code>syscw</code> — the "file write" system calls [S25]

One source file, `--count` on both: Windows counted 1 write operation, the Linux container counted 7 [S37].

[S23] `IO_COUNTERS` (`winnt.h`) — Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters [S24] `GetProcessIoCounters` (`winbase.h`) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters> [S25] `proc_pid_io(5)` — Linux man-pages. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html [S37] Both recorded runs of `iodemo`, 2026-09-15 — `demo/iodemo-c/README.md`

Some of you run Windows. You have just watched twenty minutes of Linux, and I owe you an accurate account of why [speculative].

The counting summary is a feature of `strace`. `NtTrace`, the tracer I use on Windows in this course, is a different design and prints no such summary [speculative]. Counting was the job here, so I reached for the tool that counts.

Neither system needed a tracer for this. Both were already counting, and reading the count costs one file read on Linux and one call on Windows [inference].

Read the two definitions in the table again, because they are not the same sentence. The Linux one names system calls [S25]. The Microsoft one says "write operations performed" and never says what makes one [S23]. I come back to that on the next slide.

Both demo programs read their own count. Pass `--count`, and the program prints it at the end. On Windows the Python one calls `GetProcessIoCounters` through `ctypes`, the standard library's module for calling C functions directly. On Linux it reads `/proc/self/io`, the running process's own directory.

You do not have to write code at all. Windows lets one process read another process's count, given the right query permission [S24]. That is why a ready-made process viewer can show you the same numbers [inference]. Task Manager can add I/O columns on its Details tab — check it on your own machine rather than taking my word for it.

One measurement makes that concrete. On Windows `iodemo.exe --count` printed "write operations counted by the kernel: 1" [S37]. The same program in the Linux container printed 7 [S37]. The one write is the same, and the other six belong to the Linux loader [S37] [inference].

So the answer to "what about Windows" is not "wait for Part V". Your system already counted, and here is the field.

25 25. Counting on Windows: ask the kernel

On screen:

Part II

strace is a Linux tool [S30]. Windows accounts for a process's I/O anyway, and one documented call reads it back [S24].

[code]

- **Why not a tracer.** *NtTrace prints no summary [speculative]. The program asks the kernel [S24].*
- **What it does.** *Reads `WriteOperationCount` [S23] twice. The difference is what three lines cost.*
- **What to expect.** *One operation, not three — Python buffered them [S6], and my run measured one [S35].*

*[S24] `GetProcessIoCounters` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters> [S30] *strace* — project page. <https://strace.io/> · [S35] the run: `demo/iocount-win32/README.md`*

A few slides back you saw the Linux count under `strace -c`. The Windows half of the room still has no code. Here it is. The full file in `demo/iocount-win32/` adds the imports and the print lines I cut.

Start with the handle. A handle is what Windows gives a program to name an open object, and I compare it with a file descriptor in Part V. Here it names the process itself. The PyWin32 page documents the argument as a "Process handle as returned by `OpenProcess`" [S29]. The sample passes `win32api.GetCurrentProcess()` instead, because a program reading its own counters needs no `OpenProcess` call [inference].

Then the call. `GetProcessIoCounters` "Retrieves accounting information for all I/O operations performed by the specified process" [S24]. PyWin32 hands the result back as "a dictionary representing an `IO_COUNTERS` struct" [S29], and the sample takes `WriteOperationCount` out of it [S23].

Expect one operation. I ran this on Windows on 15 September 2026, and the counter moved by one [S35]. Python collects the three writes in one buffer [S6]. Read the script carefully: it takes the second reading after the `with` block closes the file. So that one operation covers the three writes and the close together [S35].

Three `f.write` calls in the source. One write operation in the kernel. That gap is the whole point of this part [speculative].

The lab does not need PyWin32, which is a third-party package of Windows bindings. On Windows, `iodemo.py --count` reads the same counter through `ctypes`. On Linux the same program reads `/proc/self/io` and takes the field `syscw` [S25]. Either platform answers the benchmark's question.

The count covers the whole process, and the interpreter may write before your first line runs [inference]. That is why the sample reads it twice and subtracts. On my Windows run the counter read zero before the first line [S35]. On Linux the demo program started with `syscw` already at nine [S35]. Two platforms, two start-up costs. So do not compare the absolute numbers [inference].

26 26. What the counters will not tell you

On screen:

Part II

Three limits, none visible in the count.

THE LIMIT	WHAT IT DOES TO YOUR COUNT
Not one measurement	Linux counts the <code>write(2)</code> family of system calls [S25]. Microsoft says "write operations performed" [S23].
Operations, not crossings	<code>syscw</code> counts only write operations [S25]. It misses a seek [inference].
Start-up writes count too	The count covers the process, not your loop [inference]. Compare two runs.

Part V reads a Windows trace I captured [S34]. Equal footing is a course promise, not a per-lecture one.

[S23] `IO_COUNTERS` (`winnt.h`) — Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters [S25] `proc_pid_io(5)` — Linux man-pages. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html [S34] Our Nt-Trace64 `cmd.exe` capture.

Take each limit in turn, because each one changes what you may write in the report [speculative].

First, the two fields are not one measurement. Linux names the "file write" system calls, and counts them "including when invoked by the kernel as part of other syscalls" [S25]. So the Linux number is close to a crossing count and is not identical to one [inference]. Microsoft names "write operations performed" and stops there [S23], so the Windows number I cannot characterise at all [inference].

Second, and this follows from the first: a seek is a crossing, and it is not a write. The Linux field counts calls from the write family, so it misses a seek [inference]. The random-access program seeks once per write, so its count is about half its crossings [inference]. The sequential program never seeks, so for that one the two agree [inference]. Microsoft has a separate member for the "I/O operations performed, other than read and write operations" [S23]. The page does not say a seek increments it, so neither will I.

Third, the count covers the whole process — on Linux, "the process and its waited-for children" [S25]. So take the count for each program [inference]. Then report the ratio, not either absolute number [inference].

All three limits still leave the count useful. It answers the benchmark's question, and it needs no tracer to do it [inference].

27 27. It is not one buffer — it is a stack

On screen:

Part III

We counted the crossings. Now we ask where the bytes actually are.

Fifteen minutes, and this is the structural part of the lecture.

Section two left you with a good model and one obvious question. If the library holds my bytes and hands them down, where are they afterwards? The answer is not "in a buffer". It is "in one of several, and you cannot tell which without looking".

I draw one diagram in this section. I refer back to it in the durability lecture, in the memory mapping lecture and in the crash analysis lecture. If you photograph one slide today, photograph the next one.

28 28. It is not one buffer — it is a stack

On screen:

Part III

[code]

Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages. [S13] `fsync` — The Open Group.

Four levels. Read them from the top [inference].

Level one is your program's own buffer, owned by the language runtime. Level two is the C library's buffer. Level three is the operating system's own memory for file data. Level four is the cache inside the storage device. Then, at the bottom, the medium.

Notice that levels one and two sit side by side. They are alternative routes, not a pile. A Python program uses the buffer at level one and calls the kernel directly. A C program uses the buffer at level two. I come back to this in four slides, because it is the detail people get wrong [V6].

Now read the arrows. Each one is a different call, and each one moves your bytes down exactly one step. No single call takes data from the top to the bottom.

Look at the bottom arrow. I come back to it, and I am not going to skip it.

29 29. Four levels, and who empties each one

On screen:

Part III

LEVEL	WHO OWNS IT	EMPTIED BY
1 Application	<i>the runtime's buffer [S8]</i>	<code>f.flush()</code> [S17]
2 C library	<i>the <code>stdio</code> buffer [S3]</i>	<code>fflush()</code> [S3]
3 Kernel	<i>the page cache [V32]</i>	<code>fsync()</code> , Windows <code>FlushFileBuffers()</code> [V37]
4 Device	<i>the drive's write cache [V20]</i>	<i>a cache-flush command [V33]</i>

The page cache lives in the machine's memory [V32]. It is not the disk's cache — the disk has its own, one level further down [V20].

The kernel also moves data upward. It fetches pages ahead of a sequential reader, so "a sequential read() just happens to return faster because the data was already fetched" [V34]. The same levels work in both directions [inference].

The same four levels as a table, because you will want to copy this one [speculative].

Rows one and two are the two alternatives, and the Windows half of the room should know the C runtime adds no third row. Microsoft says of the layer Python enters there: "Low-level input and output calls don't buffer or format data" [S28]. I walk that path in full three slides from now [S26] [S27].

Level three deserves its own name. The page cache is the operating system's own buffer for file data, and it lives in the machine's memory. A write that reaches it has left your program, and it has not left the machine.

Two things people get wrong. First, it is not the disk's cache. The disk has its own, and that is level four, one step further down. Two caches, two owners, two failure modes.

Second, a busy machine shows very little free memory because the page cache has grown into it. That is correct behaviour and not a leak. The memory is reclaimable the moment something else needs it.

One sentence about reading, because every demonstration today is a write. The same four levels carry data upward. Level three fetches pages ahead of a sequential reader, so the read returns from memory rather than from the device [V34].

Now look at the last row. The device has a cache of its own, and emptying that one is the least reliable step in the diagram. Hold that thought.

30 30. To flush is to cross one boundary

On screen:

Part III

The verb means one thing here: to hand a buffer's contents to the next level down.

The C library manual says it plainly: "Note that fflush() flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3].

x

Do not say

Save, commit, persist — all claim durability [inference].

✓

Say

Flush. It names one boundary [inference].

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

This is the misconception I most want to remove today, and I give it a slide of its own.

The common belief is that the verb flush means "written to disk". It does not. It means "handed to the next level down", and the manual page says so in its own words [S3].

Read the quote carefully. It flushes only the user-space buffers provided by the C library. To get data physically onto a disk, the kernel buffers must be flushed too [S3]. That is two calls, not one.

So in this course, flush is a verb with a precise meaning. It crosses one boundary. If you write "save" or "persist" in your assignment when you mean flush, I will ask which boundary you crossed.

A student who leaves believing that a program flushes all the way to the platter has learned something false. Ordinary use will not correct it, because only a power cut falsifies the belief [inference].

31 31. Wholesale down, retail up

On screen:

Part III

Every level buys a large block from the level below. It sells small pieces to the level above.

That asymmetry is why every level in this diagram holds a buffer [inference]. Each level's "unit of purchase is larger than its unit of sale" [V18].

1

Buys in blocks

4096 bytes on an ext4 file [V7].

2

Sells in bytes

Your loop writes one byte [inference].

3

Same shape elsewhere

Allocators, caches, network stacks [speculative].

Four levels is four facts, and four facts are hard to keep. One rule is easier, so here is the rule that generates all four [V18].

Each level buys wholesale from the level below and sells retail to the level above. Its unit of purchase is larger than its unit of sale. That asymmetry is why every level in the diagram holds a buffer.

Apply it to level two. Take an ext4 file, one of the common Linux filesystems. There the C library takes a buffer of four thousand and ninety-six bytes [V7]. It sells single bytes to your loop. The gap between those two units is the saving you measured in part one.

I give you the rule rather than the four facts because the rule travels. You will meet it in memory allocation, in processor caches and in network buffers.

I avoid one thing deliberately. I am not giving you a warehouse or a bucket for the page cache. Physical memory is a cache rather than a container, and a container image installs the wrong intuition [speculative].

32 32. One program, one buffer — C and Python take different paths

On screen:

Part III

Each program uses at most one buffer, and not the same one.

[code]

The C program's buffer is the C library's `stdio` [S3] [V7]. Python's is a `BufferedWriter` [S8]. `FileIO` calls `write()` itself, so no `stdio` buffer sits under Python on either platform [S26] [S27] [S28] [V6].

The four levels are the union of both paths: a program uses one of levels 1 and 2, never both [inference]. Python documents 128 kilobytes as typical [S6]. On one ext4 file the C library took 4096 bytes, its block size [V7].

[S6] `open()` — Python documentation. <https://docs.python.org/3/library/functions.html#open>

I flagged this on the diagram and I am returning to it, because it is the detail people get wrong [speculative].

If you read the four levels as a stack, you expect a Python write to fall through level 1 into level 2. It does not. CPython reimplements the buffering policy above the C library rather than inheriting it [V6]. So follow the two programs instead of the diagram.

The C program calls `fwrite`. The C library collects the bytes in its own buffer, and that is level 2. The manual calls that level "the user-space buffers provided by the C library" [S3]. When the C library hands that buffer down, the bytes cross into the kernel — the manual calls the level below "the kernel buffers" [S3].

The Python program calls `f.write`. The object `open()` handed you is a `BufferedWriter`, and its buffer is level 1. Under it sits `FileIO`, which talks to the operating system. PEP 3116 named both and specified the layers in 2007 [S8]. CPython's `FileIO.write` "Only makes one system call" and calls `_Py_write` [S26], which calls `write()` [S27]. Nothing from `stdio` sits in between [V6].

Windows users, one sentence, because I closed this two slides ago. Microsoft says of the C runtime layer Python enters on your machine: "Low-level input and output calls don't buffer or format data" [S28]. The picture is the same on both platforms [inference].

So the diagram's four levels are the union of the two paths. Each program uses at most one of levels 1 and 2, and never both [inference]. Both paths then meet in the kernel. A program opened with `buffering=0` uses neither [S6].

Here is evidence you can check yourself. Python documents a typical buffer of one hundred and twenty-eight kilobytes [S6]. On one ext4 file, a Linux filesystem, the block size is four thousand and ninety-six bytes, and the C library buffer measured the same [V7]. Two sizes, a factor of thirty-two apart, and neither appears in your source code. That is one measured machine. Do not carry the number to a machine you have not measured.

33 33. The bottom level acknowledges early too

On screen:

Part III

Level four is the uncomfortable one, and skipping it would make the diagram dishonest.

1

The device holds a buffer

"most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache" [V20].

2

The standard allows a no-op

On fsync: "a null implementation is permitted" [S13].

3

Software cannot check

No trace separates a device that flushed from one that only reported it [V5].

[S13] fsync — The Open Group. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

I could stop the diagram at three levels. I am not going to, because the fourth is where data is actually lost, and no tool in this lecture can watch it [V20] [V5].

The device has its own memory. Most spinning disks and many solid state drives report a write as complete when it lands in that memory [V20]. At that moment the bytes are still short of the platter or the flash, and a power cut in that window loses them.

And here is the limit on today's thesis, which I would rather state than have you find. A trace cannot separate a device that flushed from a device that only reported that it flushed [V5]. Everything above this level is observable with a tool you can run. This level is not.

That is why a durability lecture exists, why databases are written the way they are, and why people test storage with a power switch.

34 34. Three lines that print nothing

On screen:

Part III

A demonstration. You have the stack now, so predict this one.

[code]

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Three lines. Import, print, exit [inference]. Nothing unusual anywhere.

Same method as part one: predict first, out loud. But this time I expect most of you to be right, and that is why this sits here rather than at the beginning.

Two runs. First in a terminal. Then redirected into a file, and I show you the file.

Take thirty seconds. Write two answers. Does hello appear on the terminal, and what is in the file?

Hands up if you think the terminal shows hello. Hands up if you think the file contains hello.

Now watch. Terminal first. And now redirected.

The file is empty. Zero bytes. If you predicted that, you did not guess. You used the diagram from the start of this part, and that is exactly what this demonstration tests.

35 35. Why the file is empty

On screen:

Part III

The rule sits in the C standard, not in the kernel.

1

Mode follows destination

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].

2

*A terminal is interactive**Bytes go out per line [inference].*

3

*A file is not**Bytes wait [inference].*

4

*The exit skips cleanup**No write reaches the kernel [inference].**[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>*

Here is the explanation, and notice where it comes from [S5].

The rule is in the C language standard. It is not an operating system behaviour and it is not a Python quirk. The standard says that standard output is fully buffered if and only if the stream can be determined not to refer to an interactive device.

Then the third line ends the process without running any cleanup. The standard explicitly permits termination paths that do not close files properly. The buffer is process memory. The process is gone. The bytes went with it.

Now the sentence that connects this back to section two. A trace of this program holds no write call at all. Not a failed one. Not a short one. None. The bytes never got as far as the operating system.

That is the top of the stack, proved to exist by removing it.

36 36. What this demonstration does not prove

On screen:

*Part III**It proves level one is real. It proves nothing about levels three and four.**"the honest classroom demo is a power-cut, not a `kill -9`. Killing the process loses only the runtime's buffer; the kernel still writes back." [V14]*

1

What died

The process, and the level-one buffer with it [inference].

2

What survived

Anything already in the page cache [V14].

3

Wrong conclusion

"So flush() was enough." It was enough against this test [inference].

I have to bound that demonstration immediately, because it is easy to over-read.

Killing a process destroys that process's memory, and that is level one. Bytes the program gave to the operating system earlier sit in the page cache, and the operating system writes them back in its own time [V14]. Your process does not need to exist.

So this proves that level one exists and can lose your data. It proves nothing about levels three and four.

The honest test for the lower levels is a power cut, not a kill [V14]. Pull the plug and you find out what the page cache was holding. Nobody is doing that in this room today.

Why so careful? Because students test with a kill, watch the file survive, and conclude that the program flushed everything it needed to. That conclusion holds against a kill and not against a power failure. Same demonstration, correct conclusion — the difference is which level you claim it tested.

37 37. `ltrace`: the same idea, one layer up

On screen:

Part III

strace stops at system calls. ltrace stops at calls into shared libraries [inference].

What it does

It "intercepts and records the dynamic library calls" a process makes [S31]. It can also print "the system calls executed by the program" [S31].

Where it comes from

Juan Cespedes wrote it originally [S32]. The manual page names Petr Machata too [S31]. On Debian and Ubuntu you run `apt-get install ltrace` [S32].

How to use it

[code]

It runs on Linux only [inference].

[S31] `ltrace(1)`. <https://man7.org/linux/man-pages/man1/ltrace.1.html> [S32] `ltrace`. <https://ltrace.org/>

Here is the second tracer. It makes the next slide checkable.

What it does. Its manual page says it intercepts and records the dynamic library calls the process makes, and that it can also print the system calls underneath [S31]. The page says its use is very similar to `strace` [S31]. So this is the idea you already have, one layer up [inference].

Where it comes from, and how to install it, is on the slide [S31] [S32].

How you use it. With `-s` you get the system calls as well as the library calls, which is the option that matters here [S31]. In the lab you run both tracers on one program and watch the outputs disagree.

38 38. Which levels can you actually watch?

On screen:

Part III

I promised three of four boundaries. Here is the tool each needs.

BOUNDARY	WHAT WATCHES IT	IN THIS COURSE
<i>App → library</i>	<code>ltrace</code> , not <code>strace</code> [V10]	<i>above</i>
<i>Library → kernel</i>	<code>strace</code> attaches here [S1]	<i>today</i>
<i>Kernel → device</i>	<i>block-layer tracer</i> [V30]	<i>elsewhere</i>
<i>Device → medium</i>	<i>no system-call trace shows whether the device flushed</i> [V5]	<i>none</i>

On the first slide I said three of the four boundaries are observable. This is the slide where I show you which three, and why the fourth is not one of them [V5].

There are four boundaries in the diagram. Here is what each costs to see.

The top one needs the tool from the slide before this. My source states the pair directly: `printf` is visible to `ltrace` and write to `strace` [V10]. In our demo program the library call is `fwrite` instead. That is how you test takeaway two rather than believe it.

The second boundary is the one we have watched all hour.

The fourth is not observable from software at all.

So: three of these four you can watch, one of them with a tool you now know. The one you cannot watch is where data actually gets lost. I come back to that asymmetry in section six.

39 39. The same write, at three levels

On screen:

Part IV

Now that the levels have a reason to exist, walk down them with one small program.

Eight minutes. In section three I said the levels exist, and why. This section walks down them with a program you can compile yourself.

You write C in this programme, so I am not going to apologise for the C or detour through pointers. We read the core of it. I compile it in front of you. I run it under the tracer. Then we read its trace.

40 40. The same write, at three levels

On screen:

Part IV

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no [inference]

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] `fflush(3)` — Linux man-pages. [S6] Python documentation, `open()`. [S20] `os` — Miscellaneous operating system interfaces, `os.write()`. <https://docs.python.org/3/library/os.html#os.write>

Read the table by column first, then by row.

The Python column and the C column are two routes to the same bottom row. Python's file objects hold a buffer at level one. C's `fwrite` fills a buffer at level two. Both call `write` in the end, and `write` is the system call.

Now read the last column, because it carries the point. Each level holds a buffer, or it passes the bytes straight down. A Python file object holds one. The `os.write` function does not, because it wraps the system call directly [S20]. The levels are not decoration. They are the mechanism you measured in part one.

The bottom row introduces a term I deferred until now. A file descriptor is the small integer a Linux program uses to name an open file in system calls. You have seen one: the `open` call returned three [S1]. It appears as the first argument of every `write` to that file.

Remember the word. In section five it is the first thing that differs.

41 41. Meet the program: `iodemo.c`

On screen:

Part IV

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

[code]

It prints nothing, on purpose. A program that prints nothing adds no `write` lines of its own to the trace [inference].

Nine lines on the screen. The full file adds comments and error returns.

Step one opens a file that does not exist, and that failure is deliberate. If every call succeeds, the trace is success all the way down, and the most interesting difference between the two platforms never appears. I show you that difference in section five.

Step two opens the file we mean to write.

Step three writes fourteen bytes in three calls. The C library collects them in its own user-space buffer before any of them crosses into the kernel [S3] [S5]. Watch that number.

Step four closes the file and checks the return value of the close. Almost none of the C I have read does that [speculative]. I come back to why in section six.

The program prints nothing at all. My reason is the plain one. Every `write` line in the trace then comes from the three `fwrite` calls, and not from a print statement [inference]. A silent program also makes a trace that is easier to read [speculative].

I am compiling it now, with warnings on, and running it.

42 42. Three calls in the source. One in the trace.

On screen:

Part IV

3

fwrite calls in the source

14

bytes written in total

1

write call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source [inference]:

[code]

The 3 is the file descriptor. The 14 is all three fwrite calls, in one crossing [inference] [T2].

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

This is where takeaway two stops being a claim and becomes something on a screen.

Three fwrite calls in the source. Fourteen bytes. One write in the trace.

Point at the source. Point at the trace. There is no line-by-line correspondence between them, and there never was. What you write is a library call. What the operating system sees is a system call. Your source did not say how many system calls to make — the buffering policy in the library fixed that number [inference].

The line on the slide is a prediction. I wrote it from the source before I ran anything. Now watch the real trace. Check it against the prediction. If they differ, the prediction was wrong and the machine was not.

And notice when the write happens. Not at the third fwrite. It happens when fclose runs, because fclose hands the buffer down to the kernel. The call in your source and the call in the trace are not even at the same point in time.

43 43. `fwrite` takes a size and a count

On screen:

Part IV

Four arguments, and two of them multiply.

[code]

1

It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

2

Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] `fread(3)`, which documents `fwrite(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

A small thing that trips people. It costs thirty seconds now or twenty minutes in the lab [speculative].

The call takes four arguments: a pointer to the data, the size of one item in bytes, the number of items, and the stream. In our program I passed one and six, meaning six items of one byte each. Six and one would have written the same bytes.

Why does it matter? Two reasons.

First, the return value counts items, not bytes. The manual says the count equals the number of bytes only when the item size is one [S21]. Write six items of one byte. You get four back, and you wrote four bytes. Write one item of six bytes. You get zero back, and you wrote nothing.

Second, fewer is legal. The system call underneath may transfer fewer bytes than you asked for [S2], and the library call may report fewer items than you asked for [S21]. Two different contracts, one habit. Read the return value.

44 44. Two operating systems, side by side

On screen:

Part V

Read as a comparison. Running the tools is the first lab's job.

Twelve minutes, and some of you waited for this section.

A warning about what this is not. It is not a demonstration that one platform is better. It is not a tutorial on installing a Windows tracer. It is a reading exercise on two pieces of output. The lesson is a structural difference between the systems, not a difference in formatting.

I also need to be careful about what evidence I have. I will tell you exactly what I am showing you, where it came from, and what it cannot do.

45 45. Process Monitor: files and registry, not system calls

On screen:

Part V

It reports filesystem and registry activity [S18], which is a different cut than the system-call boundary [inference].

What it does

It shows "real-time file system, Registry and process/thread activity" [S18]. That is a filesystem-and-registry view, not a list of system calls [inference].

Where it comes from

Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. The tool "combines the features of two legacy Sysinternals utilities, Filemon and Regmon" [S18]. You download it. You can also run it from Sysinternals Live [S18].

How to use it

[code]

[S18] Process Monitor. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Two Windows tools, one after the other.

This is the first. Microsoft describes it as showing real-time file system, Registry and process activity [S18]. That is a filesystem-and-registry view of the machine, and it is not a list of the system calls a program made [inference].

Where it comes from is on the slide [S18]. The same page also links a Procmon for Linux [S18].

How you use it. It captures everything at once, so you filter by process name first. Then you read the Operation column. I expect `CreateFile` and `WriteFile` rows for our program [inference]. Check me on the lab machine.

46 46. NtTrace: a tracer for the Windows Native API

On screen:

Part V

It watches the native API, "the interface between the application space and the OS kernel" [S10].

What it does

It "provides a simple trace facility for the Windows Native API", "roughly equivalent to strace on Linux" [S10]. It attaches the way a debugger does, so it "needs no driver and no administrator rights" [V21].

Where it comes from

Roger Orr wrote it. The source is MIT-licensed, and the release ships pre-built executables [S10]. You run it directly [inference].

How to use it

[code]

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/> [S33] NtTrace source. <https://github.com/rogerorr/NtTrace>

And here is the second tool, the one we read today.

What it does. Its project page says it provides a simple trace facility for the Windows Native API, and that it is roughly equivalent to strace on Linux [S10]. That comparison is the tool author's own, so you should know whose sentence it is. The same page says the tool uses the Windows debug interface to intercept the returns from the native API [S10]. The consequence, from my reading note: it needs no driver and no administrator rights [V21]. That one choice is why it runs in a teaching lab at all.

Where it comes from. Roger Orr wrote it, and the project page lists releases from 2011 to February 2026 [S10]. The source is MIT-licensed [S33].

How you use it. The page's own example filters on the substring `File`, which is the form we want [S10]. One honest word: this tool is one person's work, and Microsoft calls those entry points "subject to change from one release of Windows to another" [S9].

47 47. Why NtTrace and not Process Monitor

On screen:

Part V

Two Windows tools, two layers. One answers this lecture's question.

Process Monitor

1

Shows "real-time file system, Registry and process/thread activity" [S18]

2

The usual Windows choice [speculative]

3

The wrong layer for us [inference]

NtTrace

4

Reports native API calls, in user mode [S10]

5

Runs on the debugger interface, so it "needs no driver and no administrator rights" [V21]

[S18] Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

You have seen both tools. The choice between them is a choice about layers.

Process Monitor reports filesystem events rather than system calls, so it watches the wrong layer here [inference]. So we use NtTrace.

Be precise, because I could overstate it. NtTrace is not at the same attachment point as the Linux tracer. The Linux tracer sits at a documented kernel boundary. NtTrace sits one level above the kernel, in a user-mode library. That gap is the lesson I want you to take from this section.

Needing no administrator rights is what makes NtTrace usable here at all. A kernel-mode tracer needs a signed driver and a boot-time change, and nobody installs one in a teaching lab.

Neither of today's tools is what people run in production, and I come back to that at the end.

48 48. Three Windows words you will need

On screen:

*Part V**Win32*

The documented interface a Windows program calls. The 32 is history, not a width [inference]. Microsoft renamed it the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

The native API

The layer below Win32, inside ntdll.dll. Microsoft calls it "internal to the operating system" [S9].

The object manager

The kernel component that names open objects. Its paths begin ??\ [V22].

[S9] wininternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/wininternl/nf-wininternl-ntquerysysteminformation> [S22] Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

Three words, thirty seconds, and then the Windows trace makes sense [speculative].

Win32 is the documented interface a Windows program calls, and the Windows equivalent of the C library plus the POSIX calls. The name is historical. Microsoft renamed the same thing the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

So the 32 in the older name tells you nothing about the width of a program or of a trace [inference]. The trace you see next came from a sixty-four bit process [S36], and the name Win32 still applies to it [inference].

The native API is the layer below Win32. It lives in a library called ntdll, and it is the last user-mode code before the kernel. Microsoft's own documentation calls it internal to the operating system.

The object manager is the kernel component that names open things — files, processes, events. Its paths look different from the paths you type. You see one later in this section, in a trace of the Windows command shell [S34].

49 49. A real Windows trace

On screen:

Part V

You can already read this. The anatomy matches the Linux line, and the vocabulary differs [inference].

[code]

iodemo.exe under NtTrace, 2026-09-15 — the same four steps as the Linux trace [S36] [S37]. Ignore every argument except the handle, the file name and the return value [inference].

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

This is our own program on Windows. I traced `iodemo.exe` with `NtTrace64.exe` on 15 September 2026, and I traced the same program on Linux the same day [S36] [S37].

Read them as the four steps from part four [S36].

Name, arguments in parentheses, return value after the arrow. Same anatomy as the one Linux line I made you read.

One thing does differ, so I say it now. Here `=> 0` means success, and a return value starting `0xc` is a failure code [S36] [inference].

The three dots mark arguments I cut [S36].

The capture runs to 45 lines [S36]. These are the last four traced calls. Every call above them is the loader's, opening its own files before `main` ran [S36] [inference].

The first call wraps over two printed lines, so count calls and not lines. Call one is the deliberate failure, and its return value carries two error codes [S36]. Call two opens `iodemo.out` and returns handle `0x124` [S36]. Call three writes `Length=0xe` bytes, which is fourteen — three `fwrite` calls in one crossing [S36] [inference]. Call four closes the handle [S36].

One argument looks unfamiliar. `ObjectAttributes=0x58:"iodemo.out"` means the program named the file relative to an open directory [S36] [inference]. So neither of our two opens carries the object-manager prefix [S36]. The loader's calls above do carry it [S36]. The one I show you comes from the shell [S34].

50 50. A handle is not a file descriptor

On screen:

*Part V**First difference, visible in the first argument of almost every line.**Linux**A file descriptor. A small integer indexing a per-process table. Our trace shows = 3 [S37].**Windows**A handle. An opaque value naming an open object. Our trace shows FileHandle=0x124 [S36].**Same job in a trace, different mechanism in the system. Do not use one word for the other [inference].**[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-
IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/
iodemo-c/README.md*

First difference, and it sits in the first argument of nearly every line on both systems [S36] [S37].

One program, one file, two systems. Linux answered `3`, and Windows answered `0x124` [S36] [S37].

On Linux you get a file descriptor. A small integer indexing a table that belongs to your process. Three, four, five. You can reason about the numbers.

On Windows you get a handle. An opaque value that names an open object, and the object need not be a file. It might be a process, an event or a registry key.

In a trace they occupy the same position and do the same job, which is why it is tempting to use one word for both. Do not. In this course I hold the two words apart. If you write handle in a Linux assignment, I will ask which system you meant.

This is a small thing. I spend a slide on it because it is the cheapest instance of today's platform lesson: the same job, two mechanisms, visible in one argument.

51 51. Two error namespaces in one return value

On screen:*Part V**Second difference, and it is the layer boundary showing up where nobody expects it.*

[code]

1

First number

An NTSTATUS, the native layer's code [S36] [inference].

2

Second number

The Win32 error it translates into [S36] [inference].

3

Linux

One errno: -1 ENOENT [S37].

*[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-
IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/
iodemo-c/README.md*

Second difference, and this is the one I find genuinely interesting [speculative].

That return value is our own program's first line, the `fopen` I made fail on purpose [S36].

Look at it. It carries two error codes at once. The first is the native layer's own code. The second is the Win32 error the first translates into, with the message text you would actually see.

Two namespaces, in one return value, from one call.

Linux has one. The same failed open returns `-1` and the name `ENOENT` [S37]. That number is the error, and there is nothing to translate, because only one layer names errors.

Why does Windows have two? Because there genuinely are two layers there, and both are real interfaces with their own error vocabularies. You are looking at the seam between them, visible because the tracer sits below the translation and reports both sides.

This is why the demo program opens a file that does not exist. Without a failure this line never appears, and a clean run hides the seam between the two layers.

52 52. The stable boundary sits at a different level

On screen:

*Part V**Third difference, and the other two are symptoms of it.**In our trace of the Windows command shell a path reads "`\\??\\C:\\WINDOWS\\system32\\c_852.nls`" [S34]. That prefix "is the object-manager namespace, which a Win32 program never types" [V22].**Linux**[V8]**The system call interface is the public contract.**Syscall**Windows**[S9]**Native entry points are "subject to change from one release of Windows to another".**Win32**[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>*

Third difference, and the other two are symptoms of this one [inference].

Look at the path on this slide. It comes from the shell's trace [S34]. Our own program's two opens name their file relative to an open directory, so neither of them carries the prefix [S36].

The path starts with a backslash, two question marks, a backslash. No Win32 program ever types that string. It is the name of the file in the object manager's namespace, which sits below the interface a Windows programmer uses. The tracer is showing you a layer programs do not normally see.

Microsoft says so in its own documentation. The native entry points are internal to the operating system and subject to change between releases [S9]. That is a first-party statement.

On Linux the opposite holds. The system call interface is the public contract. You may write a system call number down and rely on it.

Now the careful sentence, and it is the takeaway. By nominate I mean one thing: the vendor documents that layer as the one it will not break. Both systems are stable at the layer they nominate, and the disagreement is about which layer that is. Everything else follows from that single placement, including where a tracer can attach.

53 53. Where the boundary sits is contested history

On screen:

Part V

Two readings survive the evidence. I will not pick one for you.

1

Reading A: deliberate

POSIX, OS/2 and Win32 each reached NT through the native API [S19].

2

Reading B: vestige

A reported POSIX subsystem that was "never really completed" [S15].

3

Linux had no choice

It "had to make the kernel boundary itself the stable interface" [V38].

[S15] The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brian-reiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] Inside Native Applications — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

I said the placement was a decision. Let me be more careful, because the evidence supports two readings and I would be overstating if I gave you only one [S15] [S19].

Reading A is design. NT had three operating environments: POSIX, OS/2 and Win32 [S19]. Each one served its own clients through its own interface. Each one reached NT through the native API [S19]. The source does not say where those environments ran, so neither do I.

On that reading the split is deliberate. An unstable lower layer then follows from it [inference].

Reading B is vestige. A POSIX subsystem did exist, and one practitioner account says it was built to satisfy United States government procurement rules and never finished [S15]. On that reading the split is what remained after the plan was abandoned.

Both accounts are in the literature, and I am not going to settle it in a practicum. The mechanism is settled. The intent is not.

Linux is the clearer case. It was a kernel built against a separately developed userland it did not control [V38]. It could not update the C library in lockstep, so it had to make the kernel boundary itself the stable one [V38].

54 54. Linux is the outlier, not Windows

On screen:

Part V

The tempting conclusion is backwards, and the evidence is a list of other systems.

"Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9]

1

Who declines the promise

FreeBSD, OpenBSD, NetBSD, Solaris/illumos, macOS [V43]. Read that list as five silences, not five equally unstable systems [V36].

2

Why Windows and macOS can

Both control their own C library [V38].

3

So say different

Never say worse [inference].

Now the correction to the conclusion you were probably forming [speculative].

After the last two slides it is tempting to think Windows is the odd one out. It is not. My source names the list: FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all behave like Windows here [V43]. None of them publishes a Linux-style promise about the raw system call interface [V9].

Do not read that list as five equally unstable systems [V36]. FreeBSD compiles in compatibility layers, so in practice it holds the binary interface that compiled programs depend on, while declining to promise it formally [V36]. Read the list as "nobody else promises it in the same way", not as "everybody else breaks it".

Linux is the exception. It is the only mainstream system that treats the raw system call interface as a public contract.

In this course you say the platforms are different. You do not say one is worse. If you write that, I will ask which property you are ranking them on.

55 55. One program, two kernels

On screen:

Part V

One program, two systems, traced on one day.

STEP	LINUX [S37]	WINDOWS [S36]
<i>failed open</i>	<code>openat(... "no-such-file.txt" ...) = -1 ENOENT</code>	<code>NtCreateFile(... "no-such-file.txt" ...) => 0xc0000034 [2 '...']</code>
<i>open</i>	<code>openat(... "iodemo.out" ...) = 3</code>	<code>NtCreateFile(... [0x124] ... "iodemo.out" ...) => 0</code>
<i>write</i>	<code>write(3, "one\ntwo\nthree\n", 14) = 14</code>	<code>NtWriteFile(FileHandle=0x124, ... Length=0xe ...) => 0</code>
<i>close</i>	<code>close(3) = 0</code>	<code>NtClose(Handle=0x124) => 0</code>

Same program, same fourteen bytes, one write on each side. Hold the program fixed, and what is left is the two systems [inference] [T4].

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

This is the slide the section was built for [speculative].

Read across the rows, not down. Each row is the same step of the same source file [S36] [S37].

Row one is the deliberate failure. The Linux call is `openat`, which is `open` with a starting directory [inference]. It returns `-1` and one error name [S37]. Windows returns a native code and the Win32 error it translates into [S36] [inference].

Row two is the open, and the two columns are not symmetric. Linux returns the file descriptor `3` as the return value [S37]. Windows returns the status `0` and hands the handle `0x124` back through an argument [S36] [inference].

Row three is the write. Fourteen bytes on Linux, `Length=0xe` on Windows [S36] [S37]. Three `fwrite` calls in the source, one call in each trace [inference].

One number to reconcile, from part two. Seven write operations there, one here. The other six were the Linux loader's [S37].

Now the caveat, and it is about the Windows tool rather than about this pair. NtTrace is one author's work [S10]. Microsoft documents those native entry points as subject to change between releases [S9]. So read the Windows column as the shape of the boundary, not as an interface you may depend on [inference].

My own note on the sample I taught from says "It is not a matched pair." [V23] This one is. Both captures ship in this deck, and you rerun them in the first lab.

56 56. What we have not looked at

On screen:

Part VI

One question I will not answer, and the work that follows from it.

Six minutes left, and I am not going to spend them listing topics.

I will put one question back on the screen. I will not answer it. Then I show you two consequences of the stack. You already have everything you need to see them.

57 57. `write()` returned success. The bytes are not on a disk.

On screen:

Part VI

I built the whole hour towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question. I leave it open today.

[S2] write(2) — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Read the first quote again. It comes from the manual page for the most basic write call there is. It says a successful return guarantees nothing about a disk [S2].

That should now be obvious rather than alarming, because you have the stack. A successful write means the bytes reached the next level down. Which level depends on which call you made.

The second quote is there so you do not think this is a bug someone will fix. Saltzer, Reed and Clark wrote it in 1984. A lower level may offer an incomplete version of a function as a performance improvement. The complete version needs the application at the end points. Every level in our diagram is an instance of that.

So: when is it actually safe? I am not answering today. I want you to leave with the question open, because a programmer who believes it is already answered writes no durability step at all.

58 58. Buffering moves your errors

On screen:

Part VI

The error arrives somewhere other than where you caused it.

1

Why it moves

The bytes only reached a buffer [inference].

2

Where it arrives

Where the library flushes, so at fclose [inference].

3

The rule

Check the return value of the call that flushes [inference].

4

The idiom hides it

"the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4].

Here is the practical consequence, and it will cost you a debugging afternoon at some point [speculative].

Your `fwrite` returned success because the bytes reached a buffer. A full disk, an unplugged device, an exceeded quota — no error from any of those has reached the library yet, and none can [inference].

That is why the demo program checks the return value of `fclose`. Almost none of the C I have read does [speculative]. That single check is the difference between having written fourteen bytes and believing you did.

The last row is worth saying slowly. The most common way to write a file in every mainstream language contains no durability step anywhere [V4].

Python differs here. A Python with `block` does raise if the close fails. The two languages differ in what they force you to notice, which is the property I am ranking them on [inference].

This is error reporting, not error recovery. Recovery belongs to the durability lecture.

59 59. Never retry a failed close

On screen:

Part VI

The obvious repair is wrong. Three storage engines dropped the retry in 2018 [V40].

"Retrying the `close()` after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

The maintainers of three databases reached the same conclusion one level down, for `fsync` : PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger [V40]. WiredTiger wrote the reason into its own source comment: "we don't retry `fsync`-style calls and panic if a flush operation fails" [V24]. Not three separate discoveries, though: that comment cites the LWN report on PostgreSQL [V24].

[S4] `close(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Now you check the return value of `close`. You will want to do the obvious thing with it, and the obvious thing is wrong [S4].

Do not retry. The manual page says why, and the reason is specific. After a failed `close` the descriptor may already be gone. If another thread has opened a file meanwhile it may have been given that same number. Your retry closes somebody else's file.

That convergence is strong, and it has a limit. The three engines have three different architectures, and their maintainers reached the same answer within months [V41]. All three replaced the retry with a deliberate crash when `fsync` fails [V40]. None of them calls again.

But they did not reach the answer separately. WiredTiger's own comment quotes the LWN report on the PostgreSQL case as its justification [V24]. So read this as three architectures agreeing, not as three independent discoveries [inference].

The transferable rule: when a failure leaves you unable to tell what state you are in, stop the program and report it. Continuing hides the damage and does not repair it.

60 60. Networking and IPC are this same mechanism

On screen:

Part VI

Not new subjects. The same objects and the same calls, wearing different names.

1

A socket on Linux

It is a file descriptor, and Linux makes that contract binding [V28].

2

A socket on Windows

An opaque SOCKET, and Windows makes Winsock the contract [V28].

3

A pipe

A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. I want to stop you filing networking under "something else".

Thirty seconds, and then I stop naming things [speculative].

When you meet networking in this course it will not be a new mechanism. On Linux a socket is a file descriptor. The same read and write calls work on it. Sockets, pipes and terminals are all file descriptors, which is why one program can wait on all of them at once [V29].

On Windows a socket is an opaque type rather than a file descriptor. The binding contract is Winsock, the Windows sockets interface, rather than the file-descriptor contract [V28]. That is section five's pattern showing up again in a place you did not expect it [V8]. Same job, different mechanism.

61 61. Honest limits

On screen:

Part VI

What today's tools cannot see.

•

The device level

[V5]

Not observable from software: no trace can check it.

•

Asynchronous I/O

[V11]

The tracer sees the setup, then no I/O [inference].

Here is what I did not establish today.

The device level is not observable from software [V5]. I said it in section three and I repeat it here. It is the limit on today's thesis, and it is where data is actually lost [V5].

Asynchronous I/O breaks the model you have just learned, and it breaks it interestingly. A modern Linux interface moves submission into shared memory rings [V11]. One option starts a kernel thread. That thread polls the rings.

Steady-state submission then makes no system calls at all. The tracer sees the setup and then nothing. The boundary you just learned to watch is already moving.

62 62. Honest limits, continued

On screen:

Part VI

And what today's tools cost.

•

strace in production

[V25]

Even the cheaper --seccomp-bpf mode [V39] "should therefore not be used on production systems!"

•

Mapping, crash dumps

—

Each has its own lecture.

And the tools. The tracer has a second and cheaper mode, selected with the option `--seccomp-bpf`. It hands the kernel a filter, so the tracer stops only for the calls you asked about. On the workload of rebuilding strace itself, plain strace costs 3.06 times the untraced run and that mode costs 1.44 times [V39].

A strace contributor disqualifies it anyway. The mode "still stops at each syscall of interest and should therefore not be used on production systems!" [V25]. Production uses a different mechanism, in-kernel programs rather than a stop at every call: eBPF on Linux, and event tracing on Windows [speculative].

I have no per-call figure for either, so I am not giving you one. Today's tools watch the layer I am teaching, which is why I use them.

Finally, memory mapping and crash dumps. Each has a lecture.

63 63. What to do on Monday

On screen:

Part VI

1

Assignment 1 measures takeaway 1

Two settings, two patterns, one mebibyte.

2

Start from section one

Add the random-access variant [inference].

3

Report your buffer size

Beside every number.

4

Do not benchmark under the tracer

Count traced, time untraced [V17].

5

Count in the dev container

Both platforms count there.

6

Week 2 opens

On 22 September.

The last thing I want you to hear is work rather than topics.

Assignment one measures takeaway one. Buffered against unbuffered, sequential against random, one mebibyte, and an explanation in Markdown. Four numbers and a paragraph.

The two programs from part one are your starting point. Add the random access variant and you have the whole experiment. You now have the model to write the explanation. I mark that part.

One thing for the Windows half of the room, because I owe you it after part two. The counting tool is Linux-only, and the assignment asks for counts. The course ships a dev container for this. It holds the same tools on a Windows machine and on a Linux one. Lecture one told you to install it. I ask nobody to produce a count their platform cannot produce.

Three rules from today that will earn you marks. First, report your version and buffer size beside every number. The default moved from eight kilobytes [S7] to a typical one hundred and twenty-eight [S6]. Second, count under the tracer [V17]. Time the program without it, in a second run [V17].

Say in your write-up that you did both. Third, your count ratio and time ratio may disagree by orders of magnitude [inference]. Explain the gap rather than assuming a mistake. The explanation carries more marks than the numbers do.

64 64. Summary — what follows

On screen:

Closing

1

[T1] Buffer for adjacent writes

A seek hands the saving back, so the access pattern sets the call count [inference].

2

[T2] Ask: accepted, or done?

A successful return reports acceptance, not completion [V12].

3

[T3] Count, do not reason

No level reports which one holds your data, so make the crossings visible [V16].

4

[T4] Say different, never worse

Each system nominates its own level as the stable one [V8].

Then name the level that holds your bytes, and measure the machine in front of you [S7].

The same four takeaways that went up at the start, in the form each one takes on Monday.

One is the graded one. Buffer when the writes are adjacent, and expect nothing from a buffer when a seek sits between every pair of writes [inference].

Two, as one question: did that call report acceptance, or completion? Success means accepted, not done [V12]. The same question covers a message broker acknowledging a message [speculative].

Three is the method of this course. No level reports which one holds your data [V16]. You have to make the crossings visible and count them.

Four is the platform claim. Say the two systems are different [V8], and if you say more, say which property you are ranking them on.

Then the line at the bottom. Name the level that holds the bytes [inference]. The one constant I could safely have quoted for sixteen years moved by a factor of sixteen in Python 3.14 [S7].

Thank you. The handout has all of this in prose with every source. Questions now, and I am here afterwards for anything that needs a terminal.

65 65. References

On screen:

- [S1] *strace(1)* — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>
- [S2] *write(2)* — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html>
- [S3] *fflush(3)* — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- [S4] *close(2)* — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>
- [S5] C99 draft, §7.19.3. <https://busybox.net/~landley/c99-draft.html>
- [S6] Built-in Functions — *open()* — Python documentation. <https://docs.python.org/3/library/functions.html#open>

- **[S7]** Increase `io.DEFAULT_BUFFER_SIZE` — CPython issue 117151. <https://github.com/python/cpython/issues/117151>
- **[S8]** PEP 3116 — New I/O (2007). <https://peps.python.org/pep-3116/>
- **[S9]** `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — Roger Orr. <https://roger-orr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S12]** Hints for Computer System Design — Lampson (1983). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- **[S13]** `fsync` — The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- **[S14]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S16]** FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf
- **[S17]** `io.IOBase.flush()` — Python documentation. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- **[S19]** Inside Native Applications — Russinovich, Microsoft Learn (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** `os` — Miscellaneous operating system interfaces, `os.write()` — Python documentation. <https://docs.python.org/3/library/os.html#os.write>
- **[S21]** `fread(3)`, which documents `fwrite(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S23]** `IO_COUNTERS` (`winnt.h`) — Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters
- **[S24]** `GetProcessIoCounters` (`winbase.h`) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiocounters>

- **[S25]** `proc_pid_io(5)` — Linux man-pages. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html
- **[S26]** `Modules/_io/fileio.c` — CPython. https://github.com/python/cpython/blob/main/Modules/_io/fileio.c
- **[S27]** `Python/fileutils.c` — CPython. <https://github.com/python/cpython/blob/main/Python/fileutils.c>
- **[S28]** Low-level I/O — Microsoft Learn, C runtime library. <https://learn.microsoft.com/en-us/cpp/c-runtime-library/low-level-i-o>
- **[S29]** `win32process.GetProcessIoCounters` — PyWin32, Mark Hammond. https://mhammond.github.io/pywin32/win32process__GetProcessIoCounters_meth.html
- **[S30]** `strace` — project page. <https://strace.io/>
- **[S31]** `ltrace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** `ltrace` — project page. <https://ltrace.org/>
- **[S33]** `NtTrace` — source repository, Roger Orr. <https://github.com/rogerorr/NtTrace>
- **[S34]** `NtTrace64` capture of `cmd.exe /c echo Hello World!` — this course (2026-09-15). <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>
- **[S35]** `iocount_win32.py`, the recorded Windows run — this course (2026-09-15). <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iocount-win32/README.md>
- **[S36]** `NtTrace64` capture of `iodemo.exe` — this course (2026-09-15). <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>
- **[S37]** `iodemo` on Linux and on Windows, the recorded runs — this course (2026-09-15). <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>
- **[V1]** A system call is not a function call — the two-term cost model.
- **[V2]** A system call is not a function call — the measured footprint of one crossing.
- **[V3]** A system call is not a function call — `pread` and `pwrite` merge the seek.
- **[V4]** A Successful `write()` Is a Handoff, Not a Save — the idiom with no durability step.
- **[V5]** A Successful `write()` Is a Handoff, Not a Save — the device level is unobservable.
- **[V6]** Standard output is buffered differently depending on where it points — CPython above `libc`.

- **[V7]** *Standard output is buffered differently depending on where it points — the measured 4096-byte buffer.*
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers — the nomination argument.*
- **[V9]** *Windows and Linux place the stable syscall boundary at different layers — Linux as the outlier.*
- **[V10]** *The System Call Boundary — ltrace against strace.*
- **[V11]** *The System Call Boundary — io_uring with a polling thread.*
- **[V12]** *Where the Abstraction Leaks — accepted, not done.*
- **[V13]** *Where the Abstraction Leaks — trust the counts, not the times.*
- **[V14]** *Where the Abstraction Leaks — a kill is not a power cut.*
- **[V15]** *Where the Abstraction Leaks — a privilege transition, not a jump.*
- **[V16]** *Where the Abstraction Leaks — count, do not deduce.*
- **[V17]** *Observing a Running System — the 442-times worst case.*
- **[V18]** *The Memory Hierarchy from Python to Physical Pages — wholesale down, retail up.*
- **[V19]** *Virtual Memory and the Page Cache — name the cache level in every number.*
- **[V20]** *File IO and Durability — the device acknowledges early.*
- **[V21]** *NtTrace — Native API tracing for Windows (Roger Orr) — a debugger, not a driver.*
- **[V22]** *NtTrace — Native API tracing for Windows (Roger Orr) — the object-manager path prefix.*
- **[V23]** *NtTrace — Native API tracing for Windows (Roger Orr) — a trace of a different program is not a matched pair.*
- **[V24]** *PostgreSQL's fsync() surprise — never retry after fsync fails.*
- **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — not for production.*
- **[V26]** *The System Call Boundary — the calls a tracer cannot see.*
- **[V27]** *A system call is not a function call — what merging the seek did to overall duration.*
- **[V28]** *Sockets and Network I-O — a Linux socket is a file descriptor, a Windows socket is not.*
- **[V29]** *Sockets and Network I-O — sockets, pipes and terminals are all file descriptors.*

- **[V30]** *File IO and Durability* — block-layer tracing watches the kernel-to-device boundary.
- **[V31]** *Observing a Running System* — the published range for the same benchmark, on other machines.
- **[V32]** *Virtual Memory and the Page Cache* — the page cache is the machine's own memory.
- **[V33]** *File IO and Durability* — the kernel issues the device cache-flush command.
- **[V34]** *Virtual Memory and the Page Cache* — readahead moves data up the same four levels.
- **[V36]** *Windows and Linux place the stable syscall boundary at different layers* — the five systems do not decline the promise equally.
- **[V37]** *File IO and Durability* — `FlushFileBuffers` is the Windows counterpart of `fsync`.
- **[V38]** *Windows and Linux place the stable syscall boundary at different layers* — why Linux could not move its own boundary.
- **[V39]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — what the cheaper `seccomp-bpf` mode costs.
- **[V40]** *PostgreSQL's `fsync()` surprise* — three engines dropped the retry in 2018.
- **[V41]** *PostgreSQL's `fsync()` surprise* — the three engines are independent of each other.
- **[V43]** *Windows and Linux place the stable syscall boundary at different layers* — the five systems that behave like Windows.
- **[V44]** *A system call is not a function call* — the 2010 counts should not be quoted as current.
- **[V45]** *A system call is not a function call* — the cost model is graded confident, not verified.
- **[V46]** *A system call is not a function call* — the measured spread in per-crossing cost across hosts.
- **[V47]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the calls that bypass `ntdll` are invisible.
- **[V48]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the debug interface serialises threads.
- **[V49]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the serialised order is the tracer's, not the program's.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

Every source behind every claim in this hour, in the order the deck used them.

The S entries are external, and every one has a link you can follow today. Most are manual pages, language documentation and standards — short, free, and written by the people who define the behaviour.

The V entries are notes from my own research vault. You cannot open those, so the handout carries a one-page summary of each together with every external source it draws on.